

* Data types & operators

Name of variable :

- must start with - or alphabets
- followed by - / alpha / digit.
- no special char allowed.

ab ---- : binary
 0 ---- : octal
 0x ---- : Hexa
 ---- : Decimal

Constant / literals :

inta = 5 ; print("%d", a)
 char ch = 'A' ;
 string s = "----" ;
 float, Double
 ↳ 6 decimal place.

Data Type :

1. int Sign (1) 11 unsigned
 10msb
 -ve +ve
 $-1x2^3 + 1x2^2 + 1x2^1 + 1x2^0$
 -2^{n-1} to $2^{n-1}-1$; n = no. of bits.
 ↳ for 4B data. (int) signed.
 $-2^{31} - 2^{31}-1$
 ↳ signed
 $-2^{32}-1$ unsigned. (%u)

Note :

- int op int = int
- int " float = float
- float op " = "

7/3 - 7/3 7/-3 - 7/3
 a:2 a:-2 a:-2 a:2
 r:1 r:-1 r:+1 r=-1

Dividend Sign = Rem Sign
 " = opp of Div

*, /, %, + follow left to Right
 +, - Associativity.

Char

1 Byte.
 Rpts: int as ASCII value.

%c for char A → 65
 %d for ASCII a → 97
 %u for 0 → 48

char ch = 'a' 3 ⇒ (c, ch) ⇒ d

C-PROGRAMMING

3. float :

- 4 Byte → single precision. (v.f)
- 8 Byte → double " (v.f, v.f)
- long double : 16 Byte. (v. Lf)
- memory :

Sign	Exponent	Mantissa
------	----------	----------

 1.023 x 10⁴
 6 digit precision.
- % do not applied on float.
- 3*2 = 6 (error: L-value)

Unary operator : → higher than binary

- applied only on variable, not constant / expression
- updates the value of var
- post ✓ pre ✓
- highest precedence.

Relational operators :

1. > >= } left to right.
 < <= }
 2. == or !=
 Right to left ⇒ { . ? :
 * 0 ⇒ false
 * Non zero ⇒ True.
 * printf(" ") ⇒ length of constant str.

3. &&, || ? Left to Right.

Short Circuit : (S.C)

- R1 && R2 ⇒ if R1 = 0 then R2 (S.C)
- R1 || R2 ⇒ if R1 = 1 then R2 (S.C)
- ⇒ S.C means not executed.
- ⇒ R1 && R2 || R3
 if R1 = 0 only this Short Circuit
- ⇒ if (R1 && R2) = 1 then R3 Short C.
- ⇒ R1 && R2 && R3 S.C if R1 = 0.

Bitwise

~, !, ^, ~, ~, <<, >>

- x >> a ⇒ Right Shift
- x << a ⇒ Left Shift
- ~x = -(x+1) [7.5] = 8

Table :

- unary +, -, (), [], . L-R
 ↳ (post fix)
- unary +, - prefix, !, ~ R-L
 *, /, sizeof
- * / % L-R
- + - L-R
- <<, >>
- <, <=, >, >= } Relational
- =, != }
 7. ==, != }
 8. & } logical
 9. ^ } Bitwise
 10. | }
 11. && } Right logic
 12. || }
 13. ?: R-L
- =, +=, -=, *=, <<=, >>= R-L
 ↳ all equal family
- ,

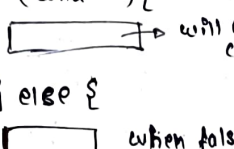
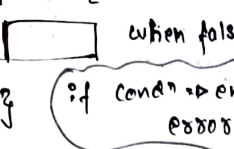
Note :

- printf("%d") → 1 } scanf("%d")
- " (" / n") → 1 } # a
- printf("----") → 3 } no. of value scanned!
- Single expression value of x updated more than once is undefined behaviour.

Ternary Operator :

- Right to left (inner first).
- (a=4) ? printf("%d", a) : printf("%d", b);
 execute first always 1.
- a = (4 ? printf("%d", a) : printf("%d", b));
 exp. 2 return 2
 a = 2;
- Short : 2B
- long int : 8B = / long long int
- unsigned / int / float / = 4B
- b = (a=5) ⇒ b = 5
 return assign value!
- x << 2 ⇒ no assign means x remain unchanged.
- x / 0 = 0 always (x ∈ int.)

* Control Flow Statement :

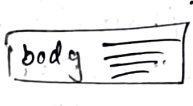
1. If (condⁿ) {

} else {

}

2. If (condⁿ) ; terminate
{

}
 3. else { } => give errors.
 : else without if.

o Switch - case : (must be int.)
Switch (a) {
case Constant-expression;
 - if break not there then below code execute until break or end of switch block.
 - default : when no case matches.
 ↳ it may not be the last case.

- case 2 :  error
 case 11 :  no sample case allowed

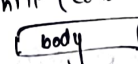
o Loop : not mandatory.
for (exp1 ; exp2 ; exp3)
{

}
 ↳ execute after body.
 ↳ must be true to execute body.

• loop run : $(UB - LB + 1)$
 $for(i = LB; i \leq UB; i++)$
 • $exp2$: empty $\Rightarrow$ condⁿ as true.
 • all exp can be empty.

* there is No Relation b/w loop & Runtime Stack.

Note:

• $int\ j = 4.0 / 2.50$ = float but store in j as int.
 • $char\ c = (b - 255) \% 256$
 • $for(i = LB; i \leq UB; i++)$
 • $for(i = UB; i \geq LB; i--)$
 • $for(i = 2; i \leq n; i *= 2) \Rightarrow \lceil \log_2 n \rceil$
 • $for(i = n; i \geq 2; i /= 2)$
 • $for(i = 1; i \leq n; i *= 2) \Rightarrow \lceil \log_2 n \rceil + 1$
 $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$

o while Loop :
while (condⁿ) {

}
 - make sure only run when condⁿ is true.
 - also called. Entry controlled loop.

o Do-while : (Exit-controlled loop)
do {

while (condⁿ)
 - run at least one time when condⁿ is false.
 ↳ starting.

3. while (condⁿ)
 ↳ can't be empty.

- $int\ i = 1$
do {
 $i++$
while (i <= 21)
 ↳ same as for loop
 ↳ same
 $while (i \leq 100)$
 $i = i + 3 \Rightarrow \frac{100 - 1 + 1}{3}$

o Break :
 ↳ only used in loop & switch.
 (Break that loop, not all)
 ↳ if (condⁿ) break;
 ↳ true $\Rightarrow$ break.

o Continue :
 ↳ only used in loop.
 ↳ skip remaining of the loop on that iteration.

• $if(i \% 2) continue;$
 $(i \% 2 = 1 \Rightarrow true)$ if odd then continue;

Note:

→ Because of the condⁿ & there effect.

* Functions & Storage Class:

o Function :
 - provide modularity & clean code.
 - $int\ fun(int, int);$ forward declⁿ
 - $fun(a, b);$ actual params: argum^{nt}
 - Control transfer to defⁿ of fun^{ct}
 - when fun^{ct} call, local variable are created, upon termⁱⁿ of fun^{ct} it is destroyed.

o Activation Record :
 - on fun^{ct} execution, compiler maintain a record; local var, return val, param^{eter}
 - these Record when exe pushed into the stack and pop on termⁱⁿation.

- main f1 f2
 (local > global)
 - fun^{ct} parameter evaluation order is not defined, it is compiler dependent.

o Static & auto :
 ↳ Static initialized only once after fun^{ct} termination.
 ↳ Static var preserve old value.

↳ auto is default storage class of local var. ↳ memory locatⁿ: stack
 ↳ scope: within the block
 ↳ lifetime: until block is executed
 ↳ initial value is garbage.
 (Static) ↳ initial value
 g/local enter program
 ↳ static is default for global.

* Storage Class:

Code Area	↳ exe ^{cut} is executed.
Static data	↳ compile time allocation
Heap	↳ runtime allocation.
Dynamic	↳ malloc(), calloc(), free()
Stack	↳ Activation Record; Dynamic

Note:
 - $for(a, b, c) \{$
 $d;$
 $\}$
 • order of exec:
 a, b, d, c
 b, d, c
 b, d, c
 d, c
 - 1, 3, 7, 15, ...
 $2^1 - 1, 2^2 - 1, 2^3 - 1, 2^4 - 1$

o Recursion:

- Tail Recursion - 
 ↳ top to bottom
 - Non Tail - 
 ↳ Bottom up.
 - If value ask, f(4), then use Bottom up approach.
 - $fun(1)$
 $fun(2)$
 $fun(3)$
 $fun(4)$
 ↳ main stack

getchar:

- ⇒ read only 1 character at a time.
- ⇒ read until Enter.

putchar:

used to write a single char.

Note:

- $s_1 == s_2$; never true b/c of add
- $*s_1 == *s_2$; (value == value)
- $\text{scanf}("%s", s)$ ⇒ read white space
- $\text{gets}(s)$ ⇒ read until '\n' ; (Enter)
- $(*p != 0)$ not null
- $\text{int d} = \text{strcmp}(b, c)$;
if (b == c)

* Structure & Union :

- user defined data type

Structure :

↳ collection of dissimilar data type.

• struct test {
int a;
char ch;
};

• struct test {
int a;
char ch;
} t1;

• typedef struct test {
int a;
char ch;
} R;

alias R → struct test

• $R * pto = \&t$;
pto → a = 14 ;
pto → b = 'B' ;

Union :

Union test {
int a;
char ch;
} t1;

Diff b/w struct & Union :

- memory allocation
struct is for all datatype.
Union is only for one which.
max (---) datatype.

• struct node *s[10];
↳ an array, each elem is a pointer of struct node.

• In output No quotation are there.

MACRO / DEFINE :

- ⇒ more efficient than function.
- ⇒ Recognized by preprocessor & just replaced by the macro body.

DEFINE max 10

DEFINE SQ(x) x * x
b = 3
a = SQ(b + 2)
∴ a = b + 2 * b + 2
= 3 + 6 + 2 = 11

* PYQ :

- infinite Recursion = abnormal termination
- " Loop = infinite Loop. for, while...
- whenever encounters point() after Recursive call put the point on to stack.
- unsigned * unsigned = unsigned.
- strcmp() → unsigned.
- in unsigned int (-ve) treated as large +ve value.